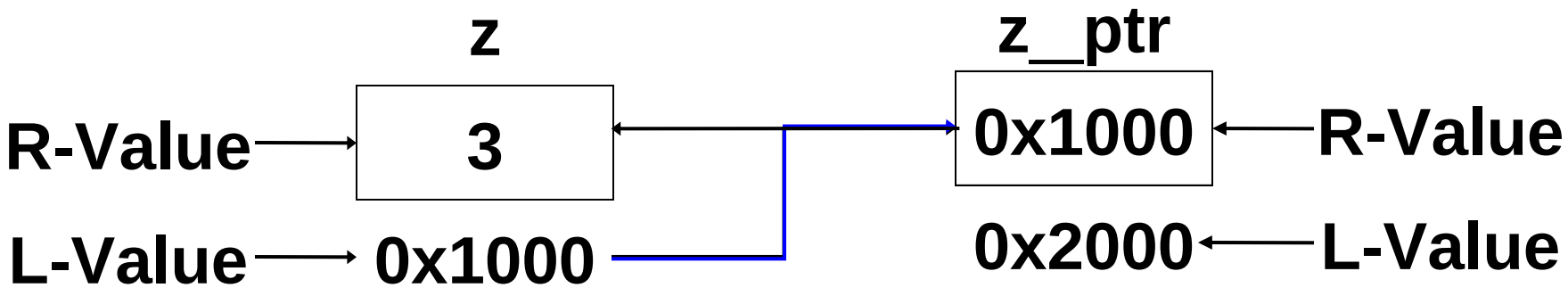
Why pointers??

- To return more than one value from a function
- To pass arrays &structure more conveniently to functions.
- Concise and efficient array access
- Access dynamic memory
- To create complex data structures
- To communicate information about memory.

What is a Pointer?

A Pointer is a variable that stores address of a memory location.



Syntax

```
int main() {
    int i = 10;
    int *pi=NULL; /*Declaration*/
    pi = &i;   /*assignment*/
    printf("%d",*pi);/*(10) accessing value*/
    *pi = 20;      /*setting value*/
    printf("%d",i);/*(20)*/
}
```

Operator & and *

- & → called as direction or reference or address of operator and used to get address of any variable

- * → called as indirection or dereference or value at operator and used to get value at the address stored in a pointer

Pointer arithmetic

- When pointer is incremented or decremented by 1, it changes by the size of data type to which it is pointing (scale factor)
- When integer ‘n’ is added or subtracted from a pointer, it changes by ‘n’ times size of data type to which pointer is pointing.
- Multiplication or division of any integer with pointer is meaning less.

Pointer arithmetic

- Addition, multiplication and division of two pointers is meaning less.
- Subtraction of two pointers have meaning only in case of arrays.

Pointer to pointer

- The pointer that stores address of another pointer variable is called as ‘pointer to pointer’.
- Example :-

```
int x=10;   int *px=&x;
int **ppx=&px;
printf(“%d”,**ppx);
```

Pass by ref

```
void sumpro(int n1,int n2, int *ps, int *pp);
int main() {
    int num1=10, num2=20, sum, pro;
    sumpro(num1,num2,&sum,&pro);
    printf("sum=%d pro=%d",sum,pro);
}
void sumpro(int n1,int n2, int *ps, int *pp){
    *ps = n1+n2;
    *pp = n1 * n2;
}
```